

Snail Avatar

汽车性能工程师专家力 AI 化通用框架

Python　多Agent架构　汽车性能开发　LLM驱动

2026年10月

✦ ✦ ✦

Snail Avatar

修炼工程师自己的"身外化身"

◈ Avatar 何意 ◈

本为宗教中"神的化身"，今指数字世界里的"数字分身"。

◈ 取名灵感 ◈

源自《凡人修仙传》——韩立以自身神识炼就"曲魂"，一具可替他征伐、代他行事的身外化身。

◈ 我们的愿景 ◈

将汽车性能工程师的专家力逐步沉淀迁移，为每位工程师炼就一具"身外化身"：替你跑仿真、查标准、算指标、答疑问，与你神识相通、协同成长。

· 蜗牛虽慢，一步一重天 ·

01 项目概述与定位

1.1 项目定位

Snail Avatar 是一个基于**多Agent架构**的电动汽车性能开发智能辅助系统，面向汽车性能开发工程师， 旨在替代一部分重复性分析工作，实现"对话式仿真 + 智能知识输出"的下一代研发模式。

性能评估

加速、最高车速、爬坡、能耗、续驶里程等指标自动计算与评估

指标管控

VTS/SSTS 整车及子系统技术规格的达标状态监控与预警

指标分析

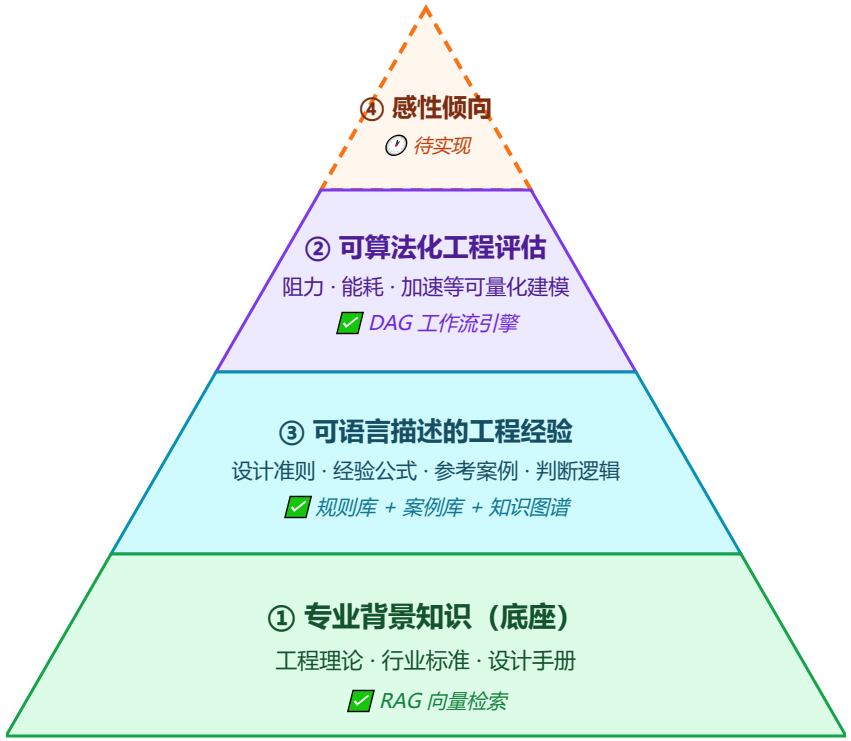
专业指标深度解读、影响因素分析、优化方向建议

知识输出

空气动力学、动力经济性等专业知识智能问答与推理

1.2 专业能力构成模型（四层能力金字塔）

资深性能工程师的专业能力呈金字塔结构：越靠底座越易编码、越靠塔尖越接近"手感"。系统自底向上逐层实现：



塔尖 • 动态反馈闭环
用户反馈采集 → 定期总结归纳 → 偏好沉淀注入；由资深工程师长期使用并持续校准

DAG 工作流引擎
私有化工程算法 + 节点化计算 + 多工况并行，自动跑动力性 / 经济性仿真

知识库引擎
规则库 + 案例库 + 知识图谱；物理公式 / 经验准则 / 对标案例精确查询

RAG 向量检索
工程书籍 / 标准 / 文档切片 + Embedding，语义检索召回

实现状态总结：金字塔下面三层（① 知识 / ② 算法 / ③ 经验）已基本落地；塔尖 ④"感性倾向"无法通过静态知识库覆盖， 需在 Agent 架构中补充动态反馈与总结机制，由资深工程师长期使用并持续校准。

1.3 技术栈（2026.10 架构升级）

类别	技术选型	说明
前端框架	Vue 3 + TypeScript + Vite	组件化 Web UI，Element Plus + ECharts，Pinia 状态管理
后端框架	FastAPI + Uvicorn	异步 API 服务，REST JSON + SSE 阶段事件流推送
LLM 服务	阿里云百炼 DashScope	Qwen3-Max（主模型）+ Qwen-Flash（摘要/路由）
向量嵌入	多模态 Embedding	qwen3-vl-embedding，支持图文混合检索
数据处理	Pandas + NumPy + openpyxl	工程计算与 Excel 车型参数表读写
配置驱动	YAML	工作流 / Agent / 节点全配置化，零代码扩展
数据存储	内存数据库 + Excel	单例模式，Excel 加载 → 内存统一访问
部署形态	前后端分离（localhost:8000 + :5173）	本地双进程启动，支持浏览器跨端访问

🔄 **架构演进：**由上一版 Flet 单机桌面应用（Python 驱动 UI）升级为 **Vue3 Web 前端 + FastAPI 后端** 的前后端分离架构， 带来浏览器跨端访问、UI 组件生态丰富、前后端并行开发、便于后续 Web 化部署等优势。

02 整体架构设计

2.1 分层架构：前后端工程外壳 + 智能内核

工程外壳 · 前后端分离

UI 交互层（Vue 3 • :5173）
Vue 3 + TypeScript + Vite | Element Plus · ECharts · Pinia · Vue Router

↕ REST JSON | SSE 事件流 ↕

API 接入层（FastAPI • :8000）
chat / process / workflows / vehicles / expert_panels / file_ops / memory_dbs 等 11 个路由

▼ 智能内核 • Snail Avatar 核心竞争力 ▼

CORE • 智能内核

Agent 调度层
MainAgent → AgentRouter（LLM 智能路由优先 + 关键词兜底）→ 各专业子 Agent 懒加载、上下文摘要传递、Function Calling

核心框架层
Agent 基类体系 · 流程引擎 Process Engine · DAG 工作流引擎 · 知识库引擎（向量/图谱/规则/案例） · 工具系统 public_functions

数据中台层
MemoryDatabase 内存单例 · Excel 车型参数持久化 · db_key 唯一标识 · 按车型 / 工作流 / 节点分层共享

2.2 已开发 Agent 体系全景

编号	Logo / 名称	类型	核心职责
a000	**MainAgent**	对话入口型	总调度、子 Agent 路由、通用问答、Function Calling
a101	**TargetDefinitionAgent**	流程驱动型	跨专业 VTS/SSTS 目标值定义全流程编排（process.yaml 驱动）
a201	**AeroExpertAgent**	知识输出型	空气动力学专业知识问答（四大知识库驱动）
a202	**PEExpertAgent**	知识输出型	动力经济性专业知识问答（四大知识库驱动）
a301	**VTS_SSTS_Agent**	数据查询型	VTS/SSTS 达标状态查询与分类汇总分析
a302	**EconomyAgent**	计算工作流型	经济性仿真计算（DAG 工作流，多温度工况）
a303	**PowerAgent**	计算工作流型	动力性仿真计算（DAG 工作流，后驱车型）
a401	**EnergyRegenerationAgent** 正在开发	计算工作流型	能量回收制动力分配计算（再生制动与摩擦制动协调分配）

📈 **体系规模：**已上线 7 个 Agent（a000/a101/a201/a202/a301/a302/a303），a401 能量回收 Agent 正在开发中，后续将按同构模式持续扩展底盘、NVH、碰撞安全等专业。

03 核心框架详解

3.1 核心框架一览



Agent 基类体系

三级继承：LLMConversationAgent → WorkflowAgentBase / ExpertAgentBase / MainAgent



智能路由

LLM 路由优先（信任度阈值）+ 关键词兜底；子 Agent 懒加载、上下文摘要传递



流程引擎

Process Engine：YAML 驱动的线性流程编排，支持多循环、条件分支、SSE 进度推送



DAG 工作流引擎

节点化计算，YAML 声明 inputs/outputs，拓扑排序调度，多工况并行



知识库引擎

向量库 + 知识图谱 + 规则库 + 案例库，统一检索 Pipeline



设计哲学

约定优于配置：YAML 单一真相源，算法文件只写纯逻辑，零代码扩展

两类引擎分工：流程引擎（Process）面向"人驱动的多步骤任务"（如目标值定义）；DAG 工作流面向"物理仿真计算"（如经济性/动力性求解）。 二者共享 Agent 基类、内存数据库与工具系统。

3.2 LLM 原生的计算流架构（核心创新）

传统数值计算只输出"数"，不输出"为什么是这个数"。LLM 拿到孤立数字无法解释、无法追问。 本项目把**语义求解**作为计算的一等公民——每个节点 **同时跑 numerical_solve() 与 semantic_solve()**，数与话同步产出。

① TimeSeries：语义随动的数据结构

不再是裸 np.ndarray，而是自带 name / unit / desc 三语义字段：

```
v = TimeSeries(time, value,
  name="车速", unit="km/h",
  desc="整车前进速度")
```

每次运算**自动传播语义**：

- v.diff() → Δv，km/h，车速的差分
- v.integrate() → ∫v，km/h • s，车速的累积积分
- v² → (km/h)²，车速的平方

数值流到哪，语义跟到哪

② 双求解器节点（CalculationNode）

每个计算节点强制两个方法，以**驾驶循环分析**为例：

```
numerical_solve()
dv = v.diff_interval()
a  = dv / dt
lth = v_avg · dt → 累加得里程
工况分段：怠速/加速/巡航/减速
```

```
semantic_solve()
自动生成 Markdown 报告：里程、时长、均速/最高速、加减速度、四种工况时间占比、计算公式、可调参数清单、输出 schema
```

算完即写报告，人不补文档

③ LLM 原生的数值求解算法

这是本项目定义的新概念：

传统： y = f(x)
黑盒，数出来就完了

LLM 原生： (y, report) = solve(x)
计算即解释，结果自带现场

LLM 读到 report 即可直接做：归因分析、DOE 变量推荐、优化方向建议。

从"人读数"到"机器读计算现场"

范式转变：传统仿真工具的输出给"人"看（一张表 + 一张图）；我们的输出给"Agent"看（数 + 语义报告）。 TimeSeries 让数据自带语义，双求解器让计算自带解释——这是面向 LLM 时代的数值计算新范式。

3.3 Process 型 Agent：从固化流程到动态规划

同一个任务"定义新车型 VTS/SSTS 目标值"——看现在怎么跑、未来怎么跑，以及两者为什么互补。

✅ 现在 • a101 按 SOP 固化流程跑

① 初始化加载数据 ↓

② LLM 检测专业顺序 ↓

🔁 **循环每个专业（气动/动力/经济/VTS...）**

分析产品 → 确定映射 → 查法规 → 定义初值 → SSTS 校核

✓ 通过→ 专业总结　　✗ 不通过→ 优化 SSTS → 专业总结

⑧ 整体总结出报告

🎯 **固化 SOP • 工程稳定性**

流程写死在 YAML，反复跑结果一致；适合有标准作业流程的任务

🔮 未来 • 用户一句话，Planner 现场规划

💬 **用户：**"帮我看看这款新车的目标值怎么定？"

↓

🧠 **Planner（LLM）现场想：**

"先看产品定义 → 对标法规 → 各专业分头算 → 校核不过就优化 → 汇总"

↓

📋 **Scheduler 派活：** a201 气动 · a202 PE · a301 VTS · a302 经济 · a303 动力

失败自动重规划，需人决策暂停问人

↓

📦 多专业目标值 + 完整分析报告

🎯 **动态规划 • 应对灵活性**

流程 LLM 现场画，每次可能不同；适合临时 / 突发 / 没 SOP 的开放任务

🔗 **共享底座（两者不变）：** Agent 基类 · 内存数据库 · 工具系统 · TimeSeries 语义数据 · 双求解器 · 知识库引擎 —— Process Engine（人写流程） 演进为 Planner+Scheduler（LLM 画流程）

两类 Agent 互补，不是替代： Process 型固化有 SOP 的长线工程任务，**保稳定性**； Planner+Scheduler 应对临时 / 突发 / 非标长线任务，**保灵活性。**

05 数据中台与 Web UI 架构

5.1 MemoryDatabase 全局单例

所有参数通过 **db_key** 唯一标识，Excel 加载后全部进入内存，Agent / 工作流 / 前端统一访问：

db_key 模式	含义
xxx_target_*	目标值
xxx_result_*	计算值
xxx_pass_*	达标标识

5.2 Web 前端界面（Vue 3 + Element Plus）



开发模式 · 对话 + 工作流 + 右侧参数面板



管理模式 · VTS 达标状态总览

06 扩展能力与设计原则

6.1 新增专业计算 Agent（7 步完成）

1. 创建目录结构
agents/a304_xxx_agent/

2. 实现计算节点
继承 CalcNodeBase

3. 节点 YAML 配置
inputs / outputs 变量声明

4. 工作流 YAML
节点 DAG 拓扑关系

5. Agent YAML 配置
agent_id / 路由关键词 / Prompt

6-7. functions + UI
工具自动扫描，UI 自动注册

核心承诺：无需修改任何核心框架代码，完全遵循开闭原则。新增专业 = 新增目录 + 新增 YAML 配置。

6.2 八大设计原则

原则	核心思想
约定优于配置	目录结构/文件名约定即配置，零代码自动发现
开闭原则	新增 Agent/面板/节点不修改核心框架代码
DRY 原则	公共函数库避免重复实现
单例模式	MemoryDatabase / ConfigManager / LLMDeploy 全局唯一
纯计算节点	工作流节点无副作用纯函数，可测试可组合
动态懒加载	Agent 实例按需加载，首次路由命中时才实例化
LLM 信任度分级	LLM 路由返回信任度，低于阈值降级关键词匹配
配置驱动	Agent 行为、工作流、节点全由 YAML 定义

07 未来规划

7.1 动态反馈与自进化机制

① **用户使用**
跑一次目标值定义
→
② **反馈采集**
显式指正 + 隐式改值
→
③ **LLM 总结经验**
归纳成结构化条目
→
④ **人审/自动分级**
晋升为正式规则
→
⑤ **下次更懂你**
经验注入会话

🔁 **使用 → 反馈 → 总结 → 沉淀 → 下次更懂你**

✅ 要做的三件事

🧠 跨会话长程记忆

记住这个项目之前定过什么、该工程师偏好保守还是激进；会话间不遗忘，长线任务跨天/跨周持续。

💬 用户反馈的总结能力

显式反馈（用户说"不对，应该..."）+ 隐式反馈（用户手动改了 Agent 输出的值）；LLM 定期把零散反馈归纳成结构化经验。

🧩 专家 Agent 自扩充规则库

自动从案例中挖规则、补法规条目、填映射表；专家能力随使用越来越强，不靠人工补文档。

⚠️ 挑战与应对

核心问题：规则库写权限开放给 LLM，怎么保证可靠性?

- **分级写权限**：阈值/权重 LLM 自动写；法规/安全红线必须人审
- **待审池**：所有自动修改先进待审池，工程师 review 后晋升
- **证据绑定**：每条规则附"依据哪次对话/哪个案例"，可追溯
- **A/B 回滚**：新规则跑对照案例，指标变差自动回退

补充：
- 🔒 幻觉防护：LLM 总结必须引用原文，不凭空造规则
- 📊 效果度量：规则更新要有"前后对比"指标
- 🚀 多用户经验联邦：脱敏后跨项目共享，A 踩坑 B 不踩
- 🔄 经验衰减：老经验定期复审，过时自动退役
- 👁 可观察学习：工程师能看到 Agent 从哪次交互学了什么

目标：让 Snail Avatar 越用越懂工程师，而不是越用越固执。

7.2 外部工具连接与仿真结果语义解析

技术原理：外部工具的语义接入

Simulink 一维仿真

整车动力经济性工况仿真（CLTC / WLTC 电耗、续航）
热管理一维仿真（电池温度、乘员舱热舒适）

StarCCM+ 三维 CFD

外流场气动仿真（Cd / Cl、风噪源）
发动机舱热管理、底盘热害分析

从"裸数据"到"工程语义"（以 CFD 为例）

500 万点裸数据
坐标 + 压力 + 速度

→ **① 参数化几何**
车长/宽高/离地间隙/格栅开口率

→ **② 语义化分区**
迎风面 / A 柱涡区 / 尾迹 / 底盘湍流

→ **③ 区域统计**
平均压力 / 最大压差 / 脉动强度

→ **④ 整车积分量**
Cd / Cl / 下压力 + 与基线 ΔCd

→ **⑤ 工程摘要**
LLM 可读 + 自动云图

Simulink 同理：上千条时间序列 → 提取工况级指标（CLTC 百公里电耗 / 电池峰值温度 / 乘员舱达温时间），而非把每秒数据喂给 LLM

工程师工作流：分析 → 改型 → 沉淀

分析时
说工程师的话

LLM 不再报告"某点静压 325 Pa"，而是说"**迎风面压损偏高、A 柱涡区压差偏大、ΔCd = +0.005**"——正是评审报告的语言。

→

改型时
闭环反问题

LLM 读完摘要直接指挥下一步——"格栅开口率 15%→12% 重跑"——自动改参数、重算、对比，形成工程闭环。

→

沉淀时
双模态文档

云图、矢量图、试验曲线是信息密度最高的部分。每份文档并行挂 LLM 可读语义；历史老报告多模态反向读图沉淀。

让 LLM 不只"算得出数"，还"读得懂仿真现场"。

08 总结：功能全景与演进路径



共享底座：TimeSeries 双求解器（LLM 原生计算流） • MemoryDatabase 内存单例 • YAML 配置驱动

从"路由专家助手"走向工程师的"身外化身"



Snail Avatar

开源 · 共建 · 共进化

框架本身只是骨架，真正的价值来自各专业工程师贡献自己的 Sub Agent.

你贡献一个专业的计算节点与规则，系统就多懂一个专业；

当气动、动力、经济、热管理、底盘、NVH ... 各专业的"身外化身"齐聚，

Snail Avatar 才真正成为资深性能工程师专家力的集体镜像.

汽车性能工程师专家力 AI 化通用框架